基於預訓練語言模型與圖神經網路之跨架構惡意軟體分析

林禹丞[1], 鄭欣明[2]

台灣科技大學資訊工程系智慧型系統實驗室[1, 2]

{m11215206、smcheng}@mail.ntust.edu.tw[1,2]

摘要

物聯網（IoT）裝置廣泛運行於異質指令集架構（ISA），使得現有惡意軟體分析方法難以跨平台通用。本文提出一種以Ghidra High P-Code爲統一中介表示之跨架構IoT 惡意軟體偵測框架，透過結合預訓練語言模型（Pre-trained Language Model, PLM）與圖卷積網路（Graph Convolutional Network, GCN），同時捕捉P-Code 指令序列之語意特徵與函式呼叫圖（Call Graph）之結構特徵。有別於先前工作以統計特徵搭配隨機森林進行分類，本研究證實純統計特徵在跨架構場景下存在顯著性能衰退，並以深度語意表示加以改善。實驗結果顯示，本方法在跨架構偵測任務上顯著優於基於統計特徵之基準方法。

關鍵字：惡意軟體分析、跨架構、Ghidra P-Code、預訓練語言模型、圖神經網路

1. 前言

物聯網（IoT）生態系統正快速擴張，全球連網裝置數量於2025 年已達211 億，預計2030 年將突破400 億[17]。這些裝置廣泛部署於智慧家庭、工業控制、醫療照護等場域，並運行於x86、ARM、MIPS、PowerPC 等異質指令集架構（ISA）。然而，受限於硬體資源與開發成本，IoT 裝置普遍缺乏完善的安全防護，成爲攻擊者鎖定的目標[14, 12]。IoT 裝置遭受的攻擊事件從2018 年的3,270 萬次急升至2022 年的1.1229 億次[10]，資安威脅日益嚴峻。

現有惡意軟體偵測系統大多依賴架構特定的資料集與手工特徵，僅適用於ARM、x86 等主流架構，對MIPS、RISC-V 等樣本稀少之平台形成偵測盲點[6]。靜態分析方法常採用可印字串（Printable String）[5]、Opcode/Byte N-gram，以及控制流程圖（CFG）或函式呼叫圖（FCG）[2, 20]等特徵，但前者具架構相依性，後者語意表達能力有限且易受混淆攻擊影響。爲此，近年研究轉向利用平台無關的中間表示（Intermediate Representation, IR），以跨越ISA 差異[11, 13]。

在中間表示的選擇上，IDA Pro Microcode[15]、Angr VexIR[1]與Ghidra P-Code[3, 8] 均爲主流方案。Ghidra 爲美國國家安全局（NSA）開發之開源逆向工程工具，其P-Code 分爲Low P-Code 與High P-Code 兩層：Low P-Code 由組合語言直接轉譯而來，保留各架構之低階操作；High P-Code 則經過去編譯分析，進一步消除暫存器差異、重建資料流與控制流，使不同ISA 下語意等價的函式趨於一致，如圖 1 所示[9]。然而，P-Code 並非完全架構無關，編譯器最佳化與連結策略仍會影響跨架構IR 之一致性[6]。

我們先前的工作[6]以Ghidra P-Code 爲基礎，抽取14 維統計特徵並搭配隨機森林分類器進行跨架構惡意軟體偵測，結果顯示單架構準確率可達99.9%，但跨架構場景下性能大幅衰退（如以MIPS 訓練、ARM 測試時僅達50.2%），揭示了純統計特徵在語意對齊上的根本局限。本研究在此基礎上，進一步引入預訓練語言模型（PLM）對High P-Code 指令序列進行深度語意編碼，並以圖卷積網路（GCN）建模函式呼叫圖之拓樸結構，以彌補統計特徵之語意缺口，提升跨架構泛化能力。

1.1 動機

儘管以P-Code 等中間表示作爲跨架構分析基礎已有初步成效，先前研究[6]指出：（1）P-Code 衍生之統計特徵仍保留ISA 層級的潛在偏差，不同架構下指令展開方式、呼叫慣例與暫存器模型的差異會影響特徵分佈；（2）編譯器最佳化（如-Os）會進一步降低跨架構函式相似度，加大遷移難度；（3）純結構特徵必須輔以語意豐富的表示，才能有效對齊不同架構下的語意等價行爲。本研究動機即在於透過深度語意模型，根本性地解決上述問題。

1.2 貢獻

本文之主要貢獻如下：

- 以先前統計特徵方法[6]之跨架構性能衰退爲出發點，系統性地分析其語意缺口，並提出以High P-Code 深度語意嵌入加以改善的框架。
- 設計P-Code 正規化前處理流程，移除Thunk 函式與外部函式，並過濾MULTIEQUAL、INDIRECT、COPY、CAST 等干擾性操作碼，提升跨架構IR 一致性。

- 結合RoBERTa 預訓練語言模型與雙層GCN，同時捕捉P-Code 指令序列之語意特徵與函式呼叫圖之結構特徵。

- 在涵蓋ARM、x86_64、MIPS 等多架構之IoT 惡意軟體資料集上進行實驗，驗證所提框架相較於基準方法之有效性。

2. 相關文獻

2.1 跨架構惡意軟體分析

靜態分析方法透過反組譯程式碼抽取特徵，無需執行二進位檔即可進行分析。早期方法以可印字串統計[5]或Opcode N-gram[14]作爲跨平台特徵，但前者語意稀疏，後者具架構相依性。圖形化方法（如CFG、FCG）雖能捕捉程式結構[2, 20]，卻對混淆攻擊較爲脆弱。Unimap[19] 將指令視爲自然語言token，透過對比式學習學習多架構指令嵌入（MAIE），將不同ISA 之語意等價指令映射至共享向量空間。VulHawk[7] 則利用熵值識別二進位檔的編譯環境，並以字串與匯入函式校正函式嵌入，應用於跨架構漏洞偵測。

2.2 中間表示方法

利用平台無關的中間表示（IR）是消除ISA 差異的主要策略之一。CFDVex[11] 以VEX IR 爲基礎，結合CFG 特徵進行IoT 惡意軟體偵測。CIMALiR[13] 以IR 進行跨平台IoT 惡意軟體聚類。我們先前的工作[6]以Ghidra P-Code 抽取14 維統計特徵，利用BinKit 2.0 資料集[4]驗證跨架構函式相似性，並以隨機森林進行分類，發現單架構準確率高達99.9%，但跨架構場景下性能嚴重衰退，揭示純統計特徵之語意對齊局限。

2.3 中間語言Ghidra P-Code

Ghidra[8] 爲NSA 開發之開源逆向工程工具，其P-Code 中間語言是一種暫存器轉換語言（Register Transfer Language, RTL），分爲Low P-Code 與High P-Code 兩層。Low P-Code 由組合語言直接轉譯，每條機器指令展開爲數個P-Code 操作，明確描述側效應與隱式狀態變更。High P-Code 則經去編譯階段產生，包含CFG 重建、堆疊框架分析、暫存器變數識別與資料流分析，進一步抽象化架構細節。如圖 1 所示，對同一C 語言函式而言，MIPS 與ARM 組合語言雖在暫存器用法、指令順序與呼叫慣例上差異顯著，但其High P-Code 表示幾乎一致，均收斂爲相同的COPY 與RETURN 操作。P-Code 之設計原則包括：顯式記憶體模型（所有記憶體存取透過LOAD/STORE 完成）、統一暫存位址空間，以及確定性語意（每個操作對抽象機器狀態具有精確定義之效果）[9]。

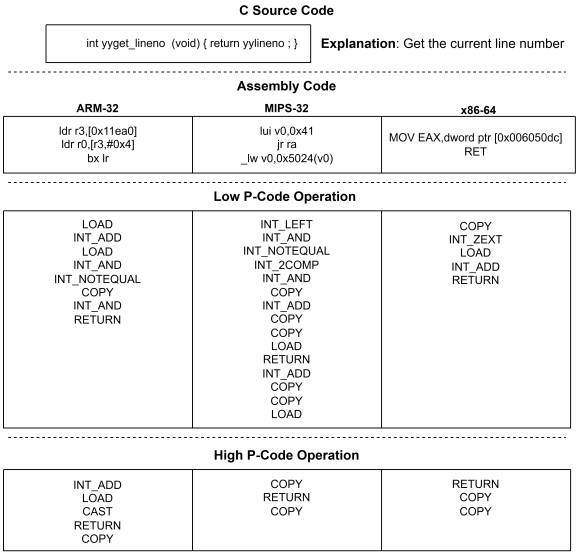

圖 1 同一C 函式在MIPS 與ARM 架構下，從組合語言經Low P-Code 轉換爲High P-Code 之過程。High P-Code 消除了架構間的語法差異，使語意等價函式趨於一致表示。

3. 方法

本節介紹所提方法之整體架構，包含預訓練語言模型、圖神經網路與對比式學習三個主要模組。圖 3 爲整體框架概覽，圖 2 爲特徵抽取流程。

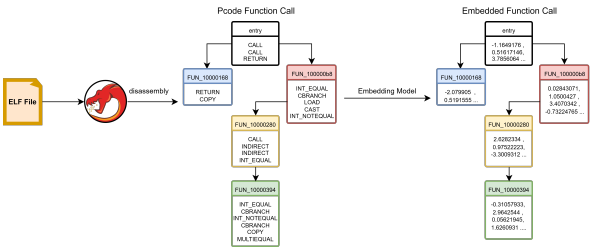

圖 2 特徵抽取流程：從ELF 二進位檔經Ghidra 轉換爲High P-Code，再進行正規化與嵌入。

3.1 預訓練語言模型

本研究採用預訓練語言模型對P-Code 指令序列進行語意編碼。透過在大規模程式碼語料庫上預訓練，模型能夠捕捉P-Code 操作碼（opcode）與運算元（operand）之語意關係，生成豐富之指令嵌入表示。

3.2 圖神經網路

將函式之控制流程圖（Control Flow Graph, CFG）轉換爲圖結構輸入，透過圖神經網路（GNN）聚合鄰居節點資訊，捕捉程式之結構特徵。每個節點代表一個基本塊（Basic Block），節點特徵由預訓練語言模型之輸出提供。

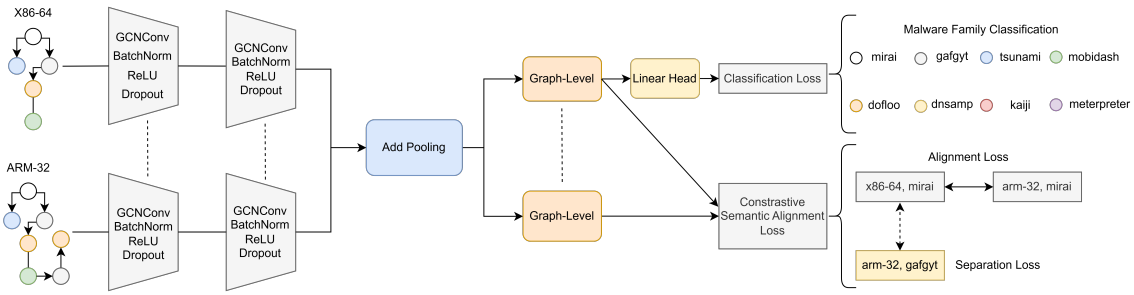

圖 3 所提跨架構惡意軟體偵測框架整體架構概覽。

3.3 對比式學習

爲強化跨架構語意對齊，本研究引入對比式學習框架。將同一惡意軟體樣本於不同架構下之表示視爲正樣本對，不同樣本之表示視爲負樣本對，透過對比損失函式縮短正樣本間之距離並拉大負樣本間之距離。

4. 實驗

在研究的實驗階段，我們將提出的方法與其他跨架構惡意軟體分析論文進行比較，並使用少量標記資料訓練經典的機器學習模型作爲基準。同時，我們也針對不同的詞向量方法與圖神經網路架構進行消融比較。

4.1 實驗設置

實驗環境爲配備NVIDIA GeForce RTX 5070 Ti（顯存16,303 MiB）之工作站，作業系統爲Ubuntu 24.04.3 LTS，CUDA 版本13.0。

預訓練語言模型（RoBERTa）：採用6層Transformer 架構，隱藏維度256，8 個注意力頭，前饋層維度1024，最大序列長度512。以遮罩語言模型（MLM）任務預訓練，遮罩比例15%，Adam 優化器，學習率1e-4，Warmup 10,000 步，線性排程器，批次大小64，訓練100個epoch。

圖卷積網路（GCN）：採用2 層架構，隱藏維度256，Dropout 0.2，加法池化（Add Pooling）。Adam 優化器，學習率0.001，批次大小32，最多訓練200 個epoch，Early Stopping（patience=15），損失函式爲Cross Entropy。

4.2 資料集

本研究收集涵蓋ARM-32、ARM-64、Intel i386、MIPS、PowerPC（PPC）及AMD x86-64 共六種架構之IoT 惡意軟體樣本。所有樣本透過VirusTotal[18] 進行多引擎掃描，並以AVClass[16] 自動化工具統一標記惡意軟體家族，最終涵蓋十個家族，原始資料集共36,078 個樣本，詳細分布如表 1 所示。

在前處理階段，我們首先移除加殼樣本，因打包技術會掩蓋惡意軟體的固有特徵並扭曲分類結果；其次移除重複樣本，以防止訓練與測試階段之間發生資料洩漏。此外，部分二進位檔因逆向工程失敗而被排除：逾時245 筆、節區格式錯誤1,432 筆、無邊1,017 筆、空節點72 筆，共計移除2,766 筆。最終有效資料集共33,312 個樣本。

4.3 基準方法

本研究與以下基準方法進行比較：

- **方法A**：XXX
- **方法B**：XXX
- **方法C**：XXX

4.4 實驗結果

實驗結果如表 ?? 所示。本方法在準確率（Accuracy）、精確率（Precision）、召回率（Recall）與F1 分數等指標上均優於各基準方法，驗證了所提框架之有效性。

5. 結論

本文提出一種結合Ghidra P-Code 中間語言、預訓練語言模型、圖神經網路與對比式學習之跨架構惡意軟體分析框架。透過將不同架構之程式碼統一轉譯爲P-Code，有效消除架構差異；並藉由深度學習模型同時建模語意與結構特徵，提升跨架構偵測能力。未來工作將探索更大規模預訓練模型之應用，以及動態分析特徵之整合。

參考文獻

[1] Angr. angr documentation. [Online]. Available: `https://docs.angr.io/en/latest/advanced-topics/ir.html`, 2026. Accessed: Feb. 2026.

[2] Q. Feng et al. Scalable graph-based bug search for firmware images. In *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, pages 480–491, 2016.

表 1 原始資料集各惡意軟體家族與架構之樣本數分布

Family	ARM-32	ARM-64	Intel	MIPS	PPC	x86_64	Total
mirai	1,062	338	1,097	1,778	3,222	1,216	8,713
gafgyt	1,062	23	1,098	1,779	3,223	1,216	8,401
tsunami	1,063	25	1,098	1,711	530	1,216	5,643
mobidash	1,063	284	1,098	307	0	1,216	3,968
dofloo	1,063	0	1,098	365	0	130	2,656
kaiji	762	187	464	619	0	429	2,461
meterpreter	138	46	604	49	24	1,195	2,056
wroba	469	86	286	232	0	79	1,152
fakebank	217	108	51	52	0	89	517
dnsamp	95	0	99	107	0	210	511
Total	**6,994**	**1,097**	**6,993**	**6,999**	**6,999**	**6,996**	**36,078**

[3] Ghidra. P-Code reference manual. [Online]. Available: `https://ghidra.re/ghidra_docs/languages/html/pcoderef.html`, 2023. Accessed: Feb. 2026.

[4] D. Kim et al. Revisiting binary code similarity analysis using interpretable feature engineering and lessons learned. *IEEE Transactions on Software Engineering*, 49(4):1661–1682, 2022.

[5] Y.-T. Lee et al. Cross platform IoT-malware family classification based on printable strings. In *Proc. IEEE 19th Int. Conf. Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 775–784, 2020.

[6] Y.-C. Lin et al. Cross-architecture IoT malware analysis using P-Code intermediate representation. In *Proc. Workshop on Recent Advances in Resilient and Trustworthy MAchine learning-driveN Systems (RTMAN)*, pages 45–50, 2025.

[7] Z. Luo et al. VulHawk: Cross-architecture vulnerability detection with entropy-based binary code search. In *Proc. NDSS*, 2023.

[8] National Security Agency. Ghidra. [Online]. Available: `https://github.com/NationalSecurityAgency/ghidra`. Accessed: Feb. 2026.

[9] N. Naus et al. A formal semantics for P-Code. In *Working Conf. Verified Software: Theories, Tools, and Experiments*, pages 111–128. Springer, 2022.

[10] A. Petrosyan. Distribution of cyber-attacks across worldwide industries in 2023. Statista. [Online]. Available: `https://www.statista.com/statistics/1315805/cyber-attacks-top-industries-worldwide/`, 2024. Accessed: Feb. 2026.

[11] T. N. Phu et al. CFDVex: A novel feature extraction method for detecting cross-architecture IoT malware. In *Proc. 10th Int. Symp. Information and Communication Technology*, pages 248–254, 2019.

[12] S. U. Qureshi, J. He, S. Tunio, N. Zhu, A. Nazir, A. Wajahat, F. Ullah, and A. Wadud. Systematic review of deep learning solutions for malware detection and forensic analysis in IoT. *Journal of King Saud University - Computer and Information Sciences*, page 102164, 2024.

[13] M. A. Rahat et al. CIMALiR: Cross-platform IoT malware clustering using intermediate representation. In *Proc. IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 0460–0466, 2024.

[14] A. D. Raju, I. Y. Abualhaol, R. S. Giagone, Y. Zhou, and S. Huang. A survey on cross-architectural IoT malware threat hunting. *IEEE Access*, 9:91686–91709, 2021.

[15] H. Rays. IDA Pro. [Online]. Available: `https://www.hex-rays.com/products/ida/`, May 2021.

[16] M. Sebastián, R. Rivera, P. Kotzias, and J. Caballero. AVClass: A tool for massive malware labeling. In *Proc. Int. Symp. Research in Attacks, Intrusions, and Defenses (RAID)*, pages 230–253, 2016.

[17] S. Sinha. State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally. [Online]. Available: `https://iot-analytics.com/number-connected-iot-devices/`, 2024. Accessed: Feb. 2026.

[18] VirusTotal. VirusTotal – free online virus, malware and url scanner. [Online]. Avail-

able: `https://www.virustotal.com`. Accessed: Mar. 2026.

[19] J. Wang et al. Can a deep learning model for one architecture be used for others? Retargeted-Architecture binary code analysis. In *Proc. 32nd USENIX Security Symp.*, pages 7339–7356, 2023.

[20] S.-Y. Yu et al. Cfg2vec: Hierarchical graph neural network for cross-architectural software reverse engineering. In *Proc. IEEE/ACM 45th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 281–291, 2023.